



CS1002 – Programming Fundamentals

Lecture # 10
Tuesday, September 20, 2022
FALL 2022
FAST – NUCES, Faisalabad Campus

Rizwan Ul Haq

General form of a C++ program

```
// Program description
#include directives
global declarations
int main()
{
    constant declarations
    variable declarations
    executable statements
    return 0;
}
```

Basics of a Typical C++ Program Environment ...

- Standard output stream object
 - `std::cout`
 - “Connected” to screen
 - `<<`
 - Stream insertion operator
 - Value to right (right operand) inserted into output stream
- **Namespace**
 - `std::` specifies that entity belongs to “namespace” **using binary scope resolution operator(`::`)**
 - `std::` removed through use of **using** statements
- **Escape characters: `\`**
 - Indicates “**special**” character output

Hello World

```
// Hello World program
```

comment

```
#include <iostream>
```

Allows access to an I/O library

```
int main() {
```

Starts definition of special function main()

```
    std::cout << "Hello World\n";
```

output (print) a string

```
    return 0;
```

Program returns a status code (0 means OK)

```
}
```

A Simple Program: Printing a Line of Text

```
1  // C++ Program
2  // Printing a line with multiple statements.
3  #include <iostream>
4
5  // function main begins program execution
6  int main()
7  {
8      std::cout << "Welcome ";
9      std::cout << "to C++!\n";
10
11     return 0;    // indicate that program ended successfully
12 } // end function main
```

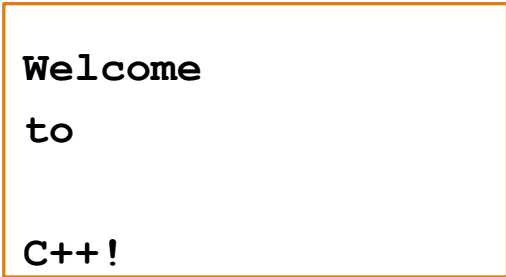
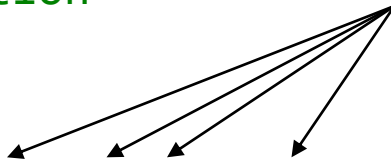
Multiple stream insertion statements produce one line of output.

Welcome to C++!

A Simple Program: Printing a text in multiple lines

```
1  //C++ Program
2  // Printing multiple lines with a single statement
3  #include <iostream>
4
5  // function main begins program execution
6  int main()
7  {
8      std::cout << "Welcome\nto\n\nC++!\n";
9
10     return 0;    // indicate that program ended successfully
11
12 } // end function main
```

Using newline characters to print on multiple lines.



```
Welcome
to
C++!
```

7 Escape Sequences

Escape Sequence	Description
\n	Newline. Position the screen cursor to the beginning of the next line.
\t	Horizontal tab. Move the screen cursor to the next tab stop (8-spaces).
\r	Carriage return. Position the screen cursor to the beginning of the current line; do not advance to the next line.
\a	Alert. Sound the system bell.
\\	Backslash. Used to print a backslash character.
\"	Double quote. Used to print a double quote character.

C++ comments

- Comments appear in green in Visual C++.
- Comments are explanatory notes; they are ignored by the compiler.
- There are two ways to include comments in a program:

// A double slash marks the start of a
//single line comment

/* A slash followed by an asterisk marks the start of a multiple line
comment. It ends with an asterisk followed by a slash. */

Comments

- Comments contain text that is not converted to machine language (it's just there for humans).
- Everything after "//" is ignored by the compiler.
- Everything between "/*" and "*/" is ignored.
- Comments
 - Are used to document programs
 - Improve program readability
 - Are ignored by compiler

C++ identifiers

Identifiers appear in **black** in Visual C++.

- An **identifier** is a name for a variable, constant, function, etc.
- It consists of a letter followed by any **sequence of letters, digits, and underscores**
- Must Begin with a **Letter Or Underscore**
- Examples of valid identifiers: First_name, age, y2000, y2k
- Examples of **invalid identifiers**: 2000y
- Identifiers cannot have special characters in them. For example: X=Y, J-20, ~Ricky, *Michael are invalid identifiers
- Identifiers are case-sensitive. For example: Hello, hello, WHOAMI, WhoAml, whoami are unique identifiers

C++ identifiers

Variables

- Variable names
- Valid identifier
 - Series of characters (letters, digits, underscores)
 - Cannot begin with digit
 - Case sensitive

Memory Concepts

Variables

- Location in memory where value can be stored
- Common data types
- **int** - integer numbers like 34, 79, 23167...
- **char** – characters, like 'a', '\$',...
- **double** -floating point numbers like 3.1416, 5.675, ...
- Declare variables with name and data type before use

```
int integer1;
```

```
int integer2;
```

```
int sum;
```

- Can declare several variables of same type in one declaration
 - Comma-separated list

```
int integer1, integer2, sum;
```

Example – adding 2 numbers

Peter: Hey Frank, I just learned how to add two numbers together.

Frank: Cool!

Peter : Give me the first number.

Frank: 2.

Peter : Ok, and give me the second number.

Frank: 5.

Peter : Ok, here's the answer: $2 + 5 = 7$.

Frank: Wow! You are amazing!

Ask computer to
solve the same
problem?

Problem: **Add
two Numbers**

Programmer: You

after Frank says “2”, Peter has to keep this number in his mind.

after Frank says “5”, Peter also needs to keep this number in his mind.

First number:

2

Second number:

5

Sum:

7

Basics of a Typical C++ Program Environment ...

➤ Standard input stream object

- `std::cin`

- “Connected” to screen

- `>>`

- Stream extraction operator

- Value from input stream inserted into right operand

➤ Namespace

- **`std::`** specifies that entity belongs to “namespace” **using binary scope resolution operator(`::`)**

- **`std::`** removed through use of **using** statements

The Corresponding C++ Program

```
#include <iostream>
using namespace std;
int main()
{
    int first, second, sum;
    cout << "Peter: Hey Frank, I just learned how to add"
         << " two numbers together."<< endl;
    cout << "Frank: Cool!" << endl;
    cout << "Peter: Give me the first number." << endl;
    cout << "Frank: ";
    cin >> first;
    cout << "Peter: Give me the second number." << endl;
    cout << "Frank: ";
    cin >> second;
    sum = first + second;
    cout << "Peter: OK, here is the answer:";
    cout << sum << endl;
    cout << "Frank: Wow! You are amazing!" << endl;
    return 0;
}
```

16 Memory Concepts

Variable names

- Correspond to **actual locations** in computer's memory
- Every variable has **name**, **type**, **size** and **value**
- When new value placed into variable, **overwrites** previous value

- `std::cin >> integer1;`
- Assume user entered 45

<code>integer1</code>	45
<code>integer2</code>	
<code>sum</code>	

- `std::cin >> integer2;`
- Assume user entered 72

<code>integer1</code>	45
<code>integer2</code>	72
<code>sum</code>	

- `sum = integer1 + integer2;`

<code>integer1</code>	45
<code>integer2</code>	72
<code>sum</code>	117

Adding Two Integers

= (assignment operator)

- Assigns value to variable
- Binary operator (two operands)
- Example:
 - `sum = variable1 + variable2;`

```

1  // C++ Program
2  // Addition program.
3  #include <iostream>
4
5  // function main begins program execution
6  int main()
7  {
8      int integer1; // first number to be input by user
9      int integer2; // second number to be input by user
10     int sum;      // variable in which sum will be stored
11
12     std::cout << "Enter first integer\n"; // prompt
13     std::cin >> integer1;                // read an integer
14
15     std::cout << "Enter second integer\n"; // prompt
16     std::cin >> integer2;                // read an integer
17
18     sum = integer1 + integer2; // assign result to sum
19
20     std::cout << "Sum is " << sum << std::endl; // print sum
21
22     return 0; // indicate that program ended successfully
23
24 } // end function main

```

Declare integer variables.

Use stream extraction operator with standard input stream to obtain user input.

Stream manipulator **std::endl** outputs a newline, then “flushes output buffer.”

Concatenating, chaining or cascading stream insertion operations.

Calculations can be performed in output statements: alternative for lines 18 and 20:

```
std::cout << "Sum is " << integer1 + integer2 << std::endl;
```

Program Output

Enter first integer

45

Enter second integer

72

Sum is 117

Constants

- Constants are data values that can not be changed during program execution
- Constants have type like integer, floating-point, character, string and boolean
 - `const double PI = 3.1415926536;`

Example

- Write a program that gets a length in inches from user, then determine and output the equivalent length in feet and inches. After that converts inches into centimeters
- (Hint: 12 inches in a feet , therefore 100 inches equal to 8 feet and 4 inches; 1 inch=2.54 centimeters)

To calculate the equivalent change, the program performs calculations using the values of a half-dollar, which is 50; a quarter, which is 25; a dime, which is 10; and a nickel, which is 5. Because these data are special and the program uses these values more than once, it makes sense to declare them as named constants. Using named constants also simplifies later modification of the program:

```
const int HALF_DOLLAR = 50;  
const int QUARTER = 25;  
const int DIME = 10;  
const int NICKEL = 5;
```

1. Prompt the user for input.
2. Get input.
3. Echo the input by displaying the entered change on the screen.
4. Compute and print the number of half-dollars.
5. Calculate the remaining change.
6. Compute and print the number of quarters.
7. Calculate the remaining change.
8. Compute and print the number of dimes.
9. Calculate the remaining change.
10. Compute and print the number of nickels.
11. Calculate the remaining change.
12. Print the remaining change.

```
    //Declare variable
int change;

    //Statements: Step 1 - Step 12
cout << "Enter change in cents: ";           //Step 1
cin >> change;                               //Step 2
cout << endl;

cout << "The change you entered is " << change
    << endl;                                //Step 3

cout << "The number of half-dollars to be returned "
    << "is " << change / HALF_DOLLAR
    << endl;                                //Step 4

change = change % HALF_DOLLAR;               //Step 5

cout << "The number of quarters to be returned is "
    << change / QUARTER << endl;            //Step 6

change = change % QUARTER;                   //Step 7

cout << "The number of dimes to be returned is "
    << change / DIME << endl;               //Step 8

change = change % DIME;                      //Step 9

cout << "The number of nickels to be returned is "
    << change / NICKEL << endl;            //Step 10

change = change % NICKEL;                    //Step 11

cout << "The number of pennies to be returned is "
    << change << endl;                     //Step 12

return 0;
```

Questions

